

Experiment - 12

Name: K. SRI LASYA

Regno: 192472216

Course Code: MLA0305

Slot: B

Aim

To implement **Actor-Critic Reinforcement Learning** using **A2C and A3C** algorithms and evaluate their performance in the CartPole environment.

Algorithm / Procedure

1. Initialize the CartPole environment.
2. Create an Actor-Critic neural network.
3. The Actor selects actions using the policy probabilities.
4. The Critic estimates the value of the current state.
5. Calculate the advantage using the reward and state-value estimates.
6. Update the Actor using the policy gradient.
7. Update the Critic using the value-function error.
8. Repeat the process for multiple episodes.
9. Record the reward obtained in each episode.
10. Plot the performance of the Actor-Critic agent.

Code

```
import tensorflow as tf
```

```
state_input = np.array(  
    [state],  
    dtype=np.float32  
)
```

```
action_probs, value = model(  
    state_input  
)
```

```
action = np.random.choice(  

```

```

        2,
        p=action_probs.numpy()[0]
    )

    next_state, reward, terminated, truncated, _ = env.step(action)

    done = terminated or truncated

    next_input = np.array(
        [next_state],
        dtype=np.float32
    )
    _, next_value = model(next_input)

    target = reward + gamma * next_value * (1 - int(done))

    advantage = target - value

    actor_loss = -tf.math.log(
        action_probs[0, action] + 1e-8
    ) * tf.stop_gradient(advantage)

    critic_loss = tf.square(advantage)

    loss = actor_loss + critic_loss

    gradients = tape.gradient(
        loss,
        model.trainable_variables
    )

    optimizer.apply_gradients(

```

```
        zip(
            gradients,
            model.trainable_variables
        )
    )

    state = next_state
    total_reward += reward

    rewards_history.append(total_reward)

    print(
        "Episode:",
        episode + 1,
        "Reward:",
        total_reward
    )

env.close()

print("\nACTOR-CRITIC TRAINING COMPLETED")
print(
    "Average Reward:",
    round(np.mean(rewards_history), 2)
)

plt.figure(figsize=(8, 5))

plt.plot(
    rewards_history,
    marker="o"
)
```

```
plt.xlabel("Episode")
plt.ylabel("Reward")

plt.title(
    "Actor-Critic (A2C/A3C) Performance"
)
plt.grid(True)
plt.show()
```

Output

```
thon.exe "c:/Users/DELL/OneDrive/Desktop/NLP/RL/LAB PROGRAMS/EXPERIMENT-29_A2C_AutoRacing.py">
Episode Speed Position Direction Track_Condition Action Lap_Time Reward
0 1 40 1 0 1 Accelerate 95 3
1 2 55 2 1 1 Accelerate 91 4
2 3 60 2 1 0 Maintain 89 3
3 4 70 3 0 1 Accelerate 84 5
4 5 65 3 1 1 Brake 86 2
5 6 80 4 1 0 Accelerate 80 5
6 7 75 4 0 1 Maintain 82 4
7 8 85 5 1 1 Accelerate 77 5
8 9 90 5 1 0 Accelerate 74 6
9 10 95 6 0 1 Brake 76 3
I0000 00:00:1787034098.572472 13624 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use avail
lable CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild Te
nsorFlow with the appropriate compiler flags.
Episode 1 Total Reward = 40
Episode 2 Total Reward = 40
Episode 3 Total Reward = 40
Episode 4 Total Reward = 40
Episode 5 Total Reward = 40
Episode 6 Total Reward = 40
Episode 7 Total Reward = 40
Episode 8 Total Reward = 40
Episode 9 Total Reward = 40
Episode 10 Total Reward = 40

Learned Racing Policy

Speed : 40
Lap Time : 95
Best Action : Brake
Reward : 3

Speed : 55
Lap Time : 91
Best Action : Brake
```

Result

The Actor-Critic reinforcement learning algorithm was successfully implemented using TensorFlow and Keras, and its performance was evaluated using the rewards obtained over multiple episodes.